FABRIC CAPACITY INTELLIGENCE PLATFORM

Module 5 Capacity-Denial Revenue Impact Calculator

How it works, end to end — data, math, and alerts

When a customer's capacity request is denied and only fixed later — or never fixed at all — that shortfall stays invisible. It sits as an individual support ticket with no roll-up, so finance and sales leadership never see it as a business risk. This module reads those tickets, separates the ones that were denied-and-later-approved from the ones denied-and-never-fulfilled, converts each into a revenue figure, ranks regions by exposure, and posts a written recommendation into Teams for a person to approve.

STATUS	Deployed and live — weekly, Mondays 08:00 UTC
WORKSPACE	Fabric Accelerators · `f78efe69-fe5e-494b-8433-77d828725ad1`
LAKEHOUSE	Fabric_Capacity_Intelligence
DATA IN THIS DOCUMENT	Synthetic ICM extract, 60 tickets, as of 2026-01-28
REVENUE FIGURES	Illustrative — the ARR reference is placeholder data
DOCUMENT DATE	11 August 2026

1 What was built

Seven stages. Everything numeric happens in pandas and is fully deterministic; the language model touches only wording, at stage 6, and never a figure.

```
ICM extract (.xlsx / Delta) | ▼ 1. BRONZE raw rows, nothing coerced ..... 60 rows × 9 cols | ▼ 2. SILVER typed, trimmed, de-duplicated ..... 60 rows + DQ report | ▼ 3. GOLD joined to subscription/ARR ..... 60 rows × 12 cols | ▼ 4. CLASSIFY four outcomes from date comparison | └─ GATE: must score 60/60 vs labelled sample, or delivery is blocked ▼ 5. PRICE ARR × capacity share × days ÷ 365 | ▼ 6. RANK group by region, order by exposure .. 11 regions | ▼ 7. RECOMMEND one action per top-3 region (LLM wording, grounding-checked) | ▼ 8. DELIVER Teams card + Approve / Reject → held for a human
```

Where it runs

COMPONENT	IDENTITY	WHAT IT DOES
Notebook `nb_module5_capacity_denial`	`4f3bb0a5…af0a`	Runs all eight stages; writes five Delta tables
Pipeline `PL_Module5_CapacityDenialRevenueImpact`	`16a11c2b…3af5`	Triggers the notebook weekly, Mon 08:00 UTC
Key Vault `kv-fabcap-ptg`	Rg-IPTool-Dev, eastus	Holds the Teams webhook and the Foundry API key
Teams channel	`capacity-findings-test`	Receives the written finding
LLM	DeepSeek-V4-Flash (Azure AI Foundry)	Wording only — never a number

> VERIFIED END TO END
>
> The analysis produces byte-identical figures on a laptop and inside Fabric. 81 automated tests pass in under a second, and the classifier reproduces all 60 known answers in the pre-labelled sample.

2 Bronze — the raw layer

Exactly what the ICM export contained. Nothing is typed, trimmed, or corrected here, so there is always an unaltered copy to compare against when a number is questioned.

60 rows × 9 columns, every value a string. Dates are still ISO text, capacities are still text, and nothing has been dropped.

BRONZE SCHEMA — THE ICM TICKET COLUMNS

COLUMN	EXAMPLE	WHAT IT IS
`IncidentId`	677681988	The support ticket
`SubscriptionId`	48840256-5fa3-…	Which customer
`TenantId`	f79c09b4-e935-…	Their tenant
`Region`	westeurope	Azure region requested
`CurrentLimitCapacity`	4	Capacity they already had
`AdditionalLimitCapacity`	83	Extra capacity asked for
`NewLimitCapacity`	87	Capacity they ended up with
`DeniedDate`	2025-10-06T16:30:29.000Z	When it was refused (blank if never)
`ApprovedDate`	2025-11-01T03:30:29.000Z	When it was granted (blank if never)

Two reference tables arrive alongside it

SHEET	ROWS	PURPOSE
`Subscription_Reference`	26	Subscription → tier and ARR. **Placeholder data today.**
`Expected_Classifications`	60	Hand-labelled correct answer for every ticket — the classifier's exam

SUBSCRIPTION REFERENCE — TIERS PRESENT

TIER	CUSTOMERS	ARR RANGE (USD)
Enterprise	7	330,818 – 565,026
Premium	4	91,844 – 127,297
Standard	8	17,927 – 34,718
Free	5	0

> WHY FREE TIER MATTERS
>
> Five customers carry $0 ARR. They still appear in every ticket count, but contribute zero dollars of exposure. That is intentional — a free customer waiting on capacity is a real service failure but not a revenue risk, and conflating the two would inflate the headline figure.

3 Silver — the cleaned layer

Same 60 tickets, now typed and checked. Every correction is recorded rather than applied silently, so nothing disappears without appearing in a report.

What changes

OPERATION	DETAIL
Text trimmed	IDs and region names stripped of whitespace; `"nan"` and `"None"` become empty
Dates parsed	ISO strings → UTC timestamps. Unparseable becomes null, not a guess
Capacities parsed	Text → numbers; missing becomes 0
Duplicates removed	Repeated `IncidentId` — first kept, rest listed in the report
Unusable rows dropped	No region, or neither date present — nothing can be said about these
Inversions flagged, *not* dropped	Approved before denied → own category, visible downstream

Data-quality result on this extract

> CLEAN
>
> **60 of 60 tickets usable.** No duplicates, no missing regions, no rows without dates, no inverted date pairs, no negative capacities, and every subscription matched the ARR reference. Nothing was dropped.

Had anything failed, the count and the specific incident IDs would appear in the run output and in the Teams message, rather than the total quietly shrinking.

Gold — adding the business context

Silver joins to the subscription reference on `SubscriptionId`, producing the layer the analysis actually reads: **60 rows × 12 columns** — the nine ticket columns plus `SubscriptionTier`, `ARR_USD`, and the join indicator.

GOLD — THREE REPRESENTATIVE ROWS

INCIDENT	REGION	TIER	ARR (USD)	DENIED	APPROVED
603081913	eastus2	Enterprise	330,818	2025-11-27	—
613636224	centralindia	Enterprise	562,514	—	2025-09-12
677681988	westeurope	Enterprise	562,514	2025-10-06	2025-11-01

> UNMATCHED SUBSCRIPTIONS
>
> A subscription with no ARR record contributes $0 exposure rather than crashing the run. It still appears in ticket counts and is named in the data-quality report, so the gap is visible instead of being mistaken for good news.

4 The classifier — what counts as a failure

A date comparison, not a model. Two dates go in, one of five categories comes out. It is simple on purpose: simple is testable, and this decides which tickets cost money.

```
delay = ApprovedDate − DeniedDate no DeniedDate, has ApprovedDate → no_denial never refused has DeniedDate, no ApprovedDate → denied_unfulfilled ← FAILURE delay ≤ 48 hours → same_day_approved normal turnaround delay > 48 hours → denied_then_approved_late ← FAILURE delay < 0 → data_quality_error never counted, always surfaced
```

Only the two marked FAILURE carry revenue impact. Everything else scores zero and stays in the data for context.

RESULT ON THIS EXTRACT

CATEGORY	TICKETS	SHARE	COUNTS AS FAILURE
Denied, then approved late	18	30%	Yes
Denied, never fulfilled	12	20%	Yes
Approved inside the cut-off	15	25%	No
No denial on record	15	25%	No
Data quality error	0	0%	Never
Total	**60**	**100%**	**30 failures**

Where the 48-hour cut-off comes from

The labelled sample settles it. Among tickets with both dates, the `same_day_approved` group runs up to **29 hours** and the `denied_then_approved_late` group starts at **145 hours**. Nothing sits in between, so any cut-off in **[29, 145) hours** reproduces the labels exactly. 48 hours — two business days — sits comfortably inside that band and is generous to the capacity team: nothing under two days is called a failure.

> THIS BECOMES A POLICY DECISION ON REAL DATA
>
> The clean gap between 29h and 145h is an artefact of synthetic data. Real ICM tickets will fill it in, at which point the cut-off stops being data-derived and becomes a judgement owned by whoever owns the capacity SLA. It lives in `config.json`, and every output states the value it ran with.

The gate

Before any figure is published, the classifier is scored against all 60 hand-labelled answers. It currently scores **60/60**. If it ever scores less, the run still produces every artefact for debugging but **delivery is blocked** — nothing reaches a leadership channel from a classifier that just got a known answer wrong.

5 The math — turning a shortfall into dollars

Two different figures are reported because they answer two different questions. Keeping them apart is the whole point; conflating them is how a credible estimate becomes a number nobody trusts.

`ARR affected — blast radius`

The *entire* annual revenue of every customer who hit a failure, each counted once no matter how many bad tickets they had.

Answers: how much of our customer base was touched by this?

This period: $1.93M across 15 customers.

`Revenue exposure — risk-adjusted`

Only the portion of revenue genuinely at stake: the share of capacity they were missing, for only the days they went without it.

Answers: what was actually put at risk?

This period: $146K — 7.6% of the blast radius.

```
exposure = ARR × capacity_share × (days_unavailable ÷ 365) ARR = the customer's annual recurring revenue capacity_share = blocked units ÷ requested units how much they were missing requested units = CurrentLimit + AdditionalLimit days_unavailable = late approval → the delay itself unfulfilled → denial date to today, capped at 365 days
```

> WHY DIVIDE BY 365
>
> ARR is an annual figure, so it is converted to a daily rate before being multiplied by the days a customer went without capacity. A customer missing 100% of their capacity for a full year would show exposure equal to their entire ARR; missing half of it for half a year shows a quarter.

Worked example 1 — a late approval

Incident 677681988, westeurope, Enterprise customer. Denied 6 October 2025, approved 1 November 2025.

INPUT	VALUE	WHERE IT COMES FROM
Customer ARR	$562,514	Subscription reference
Capacity they had	4 units	`CurrentLimitCapacity`
Capacity requested	83 units	`AdditionalLimitCapacity`
Total requested	87 units	4 + 83
Blocked while waiting	83 units	the part they didn't have
Capacity share	95.4%	83 ÷ 87
Days unavailable	25.5	6 Oct → 1 Nov

```
exposure = 562,514 × 0.954 × (25.5 ÷ 365) = 562,514 × 0.954 × 0.0697 = $37,431
```

Read plainly: this customer was missing 95% of the capacity they needed for 25 days. Charged against their annual revenue at a daily rate, that shortfall is worth roughly $37K — the single largest exposure in the period.

Worked example 2 — never fulfilled

Incident 603961530, westeurope, same Enterprise customer. Denied 3 January 2026, still not approved. They asked for 128 units on top of 10, and ended the period with the same 10 they started with.

```
capacity_share = 128 ÷ 138 = 92.8% days_unavailable = 3 Jan → 28 Jan (as-of) = 24.9 days ← still accruing exposure = 562,514 × 0.928 × (24.9 ÷ 365) = $35,622
```

The difference from example 1 matters: a late approval's clock has *stopped*. This one is still running, and grows every day the ticket stays open.

Worked example 3 — not a failure

Incident 618092636, southcentralus. Denied and approved 22 hours later — inside the 48-hour cut-off, so it is classed `same_day_approved` and its exposure is **$0.00**. It stays in the ticket counts, contributing nothing to the ranking.

6 Ranking — which region to fix first

Every ticket's exposure is summed by region and ordered. Regions with no failures keep their row: "that region had nine requests and none failed" is an answer someone will ask for.

ALL 11 REGIONS, RANKED BY REVENUE EXPOSURE

#	REGION	FAILED	LATE	UNFULFILLED	CUSTOMERS	ARR AFFECTED	EXPOSURE	SHARE
1	westeurope	4	3	1	3	$690K	$76,309	52.1%
2	northcentralus	2	1	1	2	$439K	$42,352	28.9%
3	eastus2	5	2	3	5	$502K	$10,785	7.4%
4	centralindia	1	0	1	1	$108K	$5,856	4.0%
5	uksouth	1	1	0	1	$565K	$2,786	1.9%
6	canadacentral	5	4	1	5	$720K	$2,456	1.7%
7	northeurope	1	1	0	1	$35K	$2,268	1.5%
8	eastus	2	2	0	2	$55K	$1,759	1.2%
9	westus2	2	1	1	2	$92K	$1,289	0.9%
10	southcentralus	3	1	2	3	$35K	$599	0.4%
11	canadaeast	4	2	2	4	$108K	$10	0.0%
	Total	**30**	**18**	**12**	**15***	**$1.93M***	**$146,470**	**100%**

* Customers and ARR are de-duplicated across regions, so the totals are lower than the column sums — one customer failing in three regions is still one customer.

> WHY MORE FAILURES CAN RANK LOWER
>
> canadacentral had **5 failures** but ranks 6th at $2.5K, while westeurope's **4 failures** rank 1st at $76K. Exposure scales with customer size, the share of capacity missing, and duration — so a handful of large customers blocked on most of their capacity for weeks outranks more numerous small, brief, partial shortfalls. Ticket counts alone would have pointed the team at the wrong region.

Is it getting worse?

EXPOSURE BY MONTH OF DENIAL

MONTH	FAILED REQUESTS	EXPOSURE
2025-09	5	$4,445
2025-10	6	$38,864
2025-11	6	$10,134
2025-12	10	$56,806
2026-01	3	$36,221

December is the worst month on both counts. January looks better on volume but not on value — three failures carrying $36K, because the unfulfilled ones are still accruing.

7 The recommendation — what to actually change

One recommendation per top-3 region. Rule-based, because a number a reviewer can check beats a fluent paragraph they cannot.

Step 1 — diagnose the failure mode

PATTERN	DIAGNOSIS	LEVER
Mostly late approvals	Capacity existed; approval was slow	Raise the auto-approval threshold
Mostly unfulfilled	Requests fell out of the process entirely	Named owner + fulfilment SLA
Both	Two problems at once	Threshold *and* a fulfilment check

The distinction matters: a faster approval queue does nothing for requests that were never fulfilled, because nobody was waiting on an approval in the first place.

Step 2 — size the threshold against real request sizes

Capacity is provisioned in discrete sizes (3, 4, 8, 16, 44, 83, 128, 512 …). The module walks the ladder of request sizes that actually failed in that region and picks the smallest rung clearing 80% of them — so the recommendation is a number someone can actually provision, not an average.

Step 3 — attach the evidence

Each recommendation names its two or three worst tickets with their arithmetic, so a reviewer can disagree with the *policy* without having to re-derive the *evidence*.

The three recommendations this period

#	REGION	DIAGNOSIS	RECOMMENDED ACTION
1	westeurope $76K	Delay-dominant — 3 late, 1 unfulfilled. Median 10 days late, worst 26.	Raise the auto-approval threshold to **166 units**. Would have cleared 100% of failed requests without review.
2	northcentralus $42K	Mixed — 1 late, 1 never fulfilled.	Raise the baseline ceiling to **512 units** and add a fulfilment check on denials open past 7 days.
3	eastus2 $11K	Mixed, unfulfilled-leaning — 2 late, 3 never fulfilled.	Raise the baseline ceiling to **132 units** and add the same 7-day fulfilment check.

Where the language model fits

Every figure above is computed in pandas before the model is called. DeepSeek-V4-Flash is given the finished finding and asked only to write it up. Its output is then checked:

- Every dollar figure must already have appeared in its input, or the rewrite is discarded
- Region names must be exact Azure identifiers — `westeurope`, not "West Europe" — because someone will paste them into ICM
- Aggregates it might want, like the top-3 subtotal, are pre-computed and handed over, so it never performs arithmetic

If the check fails, or the model is unreachable, the deterministic text is posted instead and the run still succeeds. **The model cannot change a number, only how it reads.**

8 The alert — what gets sent, and to whom

The point of the module is that nobody should have to open a report and interpret it. Every Monday at 08:00 UTC the agent posts the finding into Teams by itself.

WORKFLOWS → CAPACITY-FINDINGS-TEST · MONDAY 08:00 UTC

Capacity-Denial Revenue Impact

westeurope, northcentralus, and eastus2 account for 88% of the $146K risk-adjusted revenue exposure from 30 failed capacity requests across 11 regions.

Of 60 capacity requests reviewed for the period ending 2026-01-28, 30 failed (18 approved past the 48h cut-off, 12 never fulfilled), affecting 15 customers with $1.93M ARR.

westeurope — Raise the auto-approval threshold to 166 units. 3 requests were approved a median of 10 days after denial (worst: 26 days); a 166-unit ceiling would have cleared 100% of failed requests without review.
Evidence: Incident 677681988 — 83 units requested, approved 25 days late, $37K exposure.

northcentralus — Raise the baseline ceiling to 512 units and add a fulfilment check on denials open past 7 days.

eastus2 — Raise the baseline ceiling to 132 units and add the same check.

Figures are illustrative: the subscription ARR reference is still placeholder data. Rankings hold; absolute dollars do not. This finding is pending human review; nothing has been changed.

Approve Reject

What else the run produces

DELTA TABLE	ONE ROW PER	USED FOR
`dbo.module5_tickets_classified`	ticket (60)	Audit and drill-through
`dbo.module5_region_exposure`	region (11)	The ranking; Power BI
`dbo.module5_customer_exposure`	affected customer (15)	Account follow-up
`dbo.module5_exposure_trend`	month (5)	Is this getting worse?
`dbo.module5_recommendations`	top region (3)	The human review queue

Follow-up questions in the same thread

Anyone can ask the agent directly rather than opening a dashboard. Answers come from the same computed tables that produced the card, so the chat can never contradict the message:

QUESTION	WHAT COMES BACK
"why is uksouth ranked lower?"	Its rank, exposure, failure counts, and the gap to #1, with the reason exposure scales the way it does
"which region is worst?"	Top region with its figures and the recommended action
"which customers are affected?"	The five largest by exposure, with tier and regions
"show me incident 603081913"	That ticket's full arithmetic, line by line
"how is exposure calculated?"	The formula, the cut-off, and the ARR-affected distinction

Nothing is ever executed automatically

Every card carries Approve / Reject. The module reports and recommends; it does not change a capacity policy, submit a request, or alter a ticket. Two independent safeguards sit in front of delivery:

1. **The classifier gate** — a run that fails the 60-answer exam publishes nothing
2. **The grounding check** — a write-up containing an unverifiable figure is discarded in favour of the deterministic text

9 What is real, and what is not

The machinery is production-ready. The inputs are not yet, and the difference matters before anyone acts on a dollar figure.

ELEMENT	STATUS
Classifier logic	**Real.** 60/60 on the labelled sample, boundary cases tested either side of the cut-off
Exposure formula	**Real.** Arithmetic verified against hand-computed answers
Ranking and aggregation	**Real.** Totals reconcile to the ticket table; de-duplication tested
Delivery, gate, grounding check	**Real.** Live in Fabric, verified end to end
Ticket data	**Synthetic.** 60 generated tickets mirroring the real ICM schema
ARR figures	**Placeholder.** Every output says so until the real reference lands

Three things that would change the numbers

1 · The revenue reference — changes every dollar

Rankings hold because they depend on relative customer size, which the placeholder preserves roughly. Absolute dollars do not. Worth deciding at the same time: **ARR or ACR?** Azure Consumed Revenue may be the more defensible basis, since a customer blocked on capacity cannot consume — which is the shortfall being measured.

2 · Ticket status — likely overstates exposure today

Without a status field, "denied with no approval date" cannot be separated into *rejected permanently* and *still being worked*. Both are currently counted as failures. That affects **12 of the 30** flagged requests and, because unfulfilled tickets accrue exposure every day, it inflates the headline. A status column from ICM is the single change that would most improve the honesty of this number.

3 · Multiple denial rounds

The schema carries one denial date and one approval date. If ICM records a history, a request denied twice before approval is a worse outcome than this model can currently express.

> READ THE HEADLINE THIS WAY
>
> "$146K of revenue exposure" today means: *given placeholder ARR figures and treating every open denial as a failure, this is the scale and, more usefully, this is the ordering of regions.* The ordering is the actionable part. The dollars become trustworthy when the ARR reference and the ticket status column arrive.

Policy settings, all in one place

SETTING	VALUE	EFFECT OF CHANGING IT
`meaningful_delay_hours`	48	What counts as a failure at all
`severity_medium_days`	7	When a delay stops being routine
`severity_high_days`	21	When a delay becomes a headline
`unfulfilled_cap_days`	365	How much one old open ticket can dominate
`annualisation_days`	365	ARR → daily rate conversion
`top_n_regions`	3	How many recommendations per message

All live in `config.json`. Every output prints the values it ran with, so a reader can always see the assumptions behind a figure.

Figures from the synthetic ICM extract, 60 tickets, as of 2026-01-28. Revenue figures illustrative.